Phase 5 - phase5_report

PHASE 5 — EVIDENCE INTELLIGENCE & RAG

Real Estate AI Decision Intelligence Platform — Phase Completion Report

Prepared following the Master A–Z Development, Audit & Integration Prompt, Section 19 (Phase Completion Report), Section 18 (Repository Audit Process), and matching Phase 3's code-structure convention (live PostgreSQL, `db/connection.py`, `src/<topic>/*.py` modules, `generate_*_report.py` + `export_results_csv.py` orchestrators, numbered CSVs, chart PNGs).

---

1. PHASE STATUS

PHASE 5 — COMPLETE. The Evidence Registry, chunking, hybrid (lexical + semantic) retrieval, reranking, grounding validation, prompt-injection protection, retrieval observability, and RAG evaluation are all built, wired into two orchestrators (Markdown report + numbered CSV export), and independently re-checked with a 27-test automated suite — **27/27 passing.**

---

2. OBJECTIVE

Build the trusted evidence and grounding layer *before* Phase 9's multi-agent integration (Master Prompt §5.1): *"Retrieve evidence, not merely generate text."* Every claim the platform eventually makes to a user must be traceable Source → Evidence → Claim → Decision (§3.3), and if no evidence exists, the system must say so rather than invent it (§5.12).

---

3. REPOSITORY AUDIT SUMMARY (Section 18) — What Phase 5 Found Already Waiting

Re-inspecting the Phase 1/3 deliverables specifically for Phase 5 turned up two things that shaped this phase's design:

1. **`ai.documents` already exists in the Phase 1 schema** (`sql/01_schema.sql`, lines 164–173) with exactly the evidence-source shape Section 5.2 asks for: `document_id, source, document_date, city, locality, topic, text, embedding vector(384)`. Phase 5 did not need to invent a document store — it reads this table (or, for local re-check, the identical `data/processed/documents.csv` that was loaded into it).
2. **120 real documents already loaded**, spanning 4 source types (Internal Research 37, Developer Filing 36, News 29, Market Report 18) and 6 topics (Policy/Regulatory Note, Infrastructure Update, Market Outlook, Developer Report, Investment Memo, Locality Deep-Dive), across 6 cities, dated 2024-08 to 2026-08. The `embedding vector(384)` column is present but **empty in every row** — reserved by the original schema author for a transformer-embedding model. This phase does not populate it (see §11, Limitations) — its own semantic signal (LSA) is stored separately, not written into a 384-dim column it doesn't actually fill.

No Phase 1–4 code, schema, or data was modified. Phase 5 reads `ai.documents` read-only, consistent with §17 (Change Management).

---

4. INPUTS

- `ai.documents` (live DB) / `data/processed/documents.csv` (local re-check) — 120 rows, 8 columns.
- `data/processed/localities.csv`, `cities.csv`, `properties.csv` — copied in for future cross-referencing (e.g. linking evidence to a specific property's locality) but not required by the current pipeline.
- Phase 1 schema (`sql/01_schema.sql`) — confirms `ai.documents` structure and the reserved `embedding` column.

---

5. WHAT WAS BUILT

```
phase5/
├── data/processed/             # documents.csv, localities.csv, cities.csv, properties.csv (copied in)
├── db/
│   └── connection.py           # unchanged Phase 1/3 pattern: pooled PostgreSQL, env-var only credentials
├── src/evidence/
│   ├── evidence_registry.py    # Section 5.3 -- EvidenceRegistry (DB path + documented CSV re-check path)
│   ├── chunking.py             # Section 5.4 -- sentence-aware chunker, document relationships preserved
│   ├── lexical_retrieval.py    # Section 5.5 -- BM25 from first principles
│   ├── semantic_retrieval.py   # Section 5.5 -- LSA (TruncatedSVD) -- explicitly NOT transformer embeddings
│   ├── hybrid_retrieval.py     # Section 5.5 -- normalized BM25+LSA blend, metadata filtering
│   ├── reranking.py            # Section 5.6 -- relevance + source quality + recency + query alignment
│   ├── grounding.py            # Section 5.7 -- citation ID validation, claim validation, coverage, weak-grounding flags
│   ├── prompt_injection.py     # Section 5.8 -- pattern-based injection/authority-spoofing/concealment detector
│   ├── observability.py        # Section 5.9 -- query/filters/latency/result-count/zero-result-rate logging
│   ├── rag_evaluation.py       # Section 5.10 -- 6 question categories
│   ├── generate_evidence_report.py   # orchestrator -> docs/evidence_report.md (--source db|csv)
│   ├── export_results_csv.py         # orchestrator -> docs/evidence_results_csv/*.csv (--source db|csv)
```

```
│   ├── generate_charts.py           # orchestrator -> docs/evidence_charts/*.png (--source db|csv)
│   └── test_phase5.py               # 27 automated tests (CSV path, DB-independent)
├── docs/
│   ├── evidence_report.md      # generated report (this run: --source csv)
│   ├── evidence_charts/        # 4 PNGs
│   ├── evidence_results_csv/   # 8 numbered CSVs
│   ├── phase5_report.md        (this file)
│   └── limitations.md
├── requirements-phase5.txt
└── README_PHASE5.md
```

Data source convention (matches Phase 3 exactly): every orchestrator defaults to `--source db` (live PostgreSQL via `db/connection.py`, `DB_USER`/`DB_PASSWORD` env vars) — the path intended for the user's local environment, identical in spirit to how Phase 3's `generate_eda_report.py` queries `core.*` tables directly with **"No CSV dependency"** stated in its own docstring. A `--source csv` flag was added purely so this phase could be **independently re-checked against real data in an environment without a live PostgreSQL instance** (no DB engine is available in this sandbox — see §13). The CSV path reads the *identical rows* that live in `ai.documents` (same file that was loaded into it during Phase 1's ingestion), so a re-check run and a DB run produce the same evidence registry content; only the transport differs.

---

6. RE-CHECK / VERIFICATION RESULTS (what you asked me to specifically re-confirm)

Every module was run against the real 120-document dataset, not just unit-tested in isolation:

Module	Re-check performed	Result
Evidence Registry	Loaded all 120 CSV rows, checked field completeness	120/120 loaded, all required fields present
Chunking	Ran on real data (short docs) + a synthetic 4,369-char long document	Real data: 120 docs → 120 chunks (correct no-op); synthetic: 1 doc → 20 chunks (splitter proven to work once documents grow)
BM25 (lexical)	Query `"infrastructure connectivity Pune"`	Correctly surfaced Pune infrastructure documents, ranked by term relevance
LSA (semantic)	Query `"is this locality a good investment opportunity right now"` (no exact keyword overlap with any title)	Correctly surfaced "Locality Deep-Dive" documents via co-occurrence signal alone
Hybrid retrieval	Filtered query (`city=Pune`) + a gibberish no-match query	Filter respected; no-match query returned `[]`, never invented
Reranking	Same candidate set before/after rerank	Set membership identical (only reordered); a Developer Filing with a marginally higher hybrid score was correctly outranked by an Internal Research document once source-confidence was weighted in
Grounding	4 constructed cases: well-grounded / hallucinated citation ID / misquoted number / uncited numeric claim	All 4 classified correctly (`PASS` / `BLOCKED` / `BLOCKED` / `PASS WITH LIMITATIONS`) — two bugs were found and fixed during this re-check (see §12)
Prompt injection	Real 120-doc corpus + 5-case adversarial set (3 malicious, 2 clean, synthetic)	Real corpus: 0 flags (expected — clean synthetic real-estate text). Adversarial set: 3/3 malicious flagged, 2/2 clean passed
Observability	2 queries (1 hit, 1 miss) through the logger	`zero_result_rate_pct` correctly computed as 50.0%

RAG evaluation	6 categories run against the live retriever	5/5 scored categories passed (100%); `unsupported` category correctly left unscored by design

Automated suite: `test_phase5.py` **— 27/27 tests passing** after the fixes below.

---

7. TESTS EXECUTED / PASSED / FAILED

Initial run: **26/27 passed, 1 failed** (`test_chunking_never_splits_a_sentence`) — investigated and found to be an **overly strict test**, not a chunker bug: the overlap mechanism deliberately carries a trailing text fragment from the previous chunk forward for context continuity, and that fragment is not required to be a whole sentence. The test was corrected to assert the actually-important invariant (every original sentence appears intact, verbatim, in at least one chunk) rather than "no fragment ever appears anywhere." Final run: **27/27 passed.**

---

8. BUGS FOUND AND FIXED DURING RE-CHECK

This section exists because you specifically asked for a re-check — these are the real issues it caught:

1. **Grounding claim-extraction bug:** citation markers like `[ev_1_0]` contain digits (`1`, `0`), and the numeric-claim regex was matching those digits as if they were factual claims, causing well-grounded answers to be wrongly flagged as unsupported. **Fixed** by stripping citation markers from the sentence before extracting numeric claims (`grounding.py`, `validate_claims`).
2. **Report-generator demo-text construction bug:** the demo "well-grounded answer" in `generate_evidence_report.py` was built by naively splitting evidence text on `.`, which also splits on the decimal point inside numbers like `59.9` — truncating the claim to `59` and making it fail its own grounding check. **Fixed** by using the same `split_sentences()` function the grounding validator itself uses, so the demo text is constructed the same way a real answer would be.
3. **Sentence-boundary fragmentation when appending a citation after a period:** appending `" [ev_1_0]"` after a sentence that already ends in `.` created a second pseudo-sentence containing only the citation, orphaning it from the numeric claim it was meant to support. **Fixed** by stripping the trailing period before appending the citation, then adding one final period.

All three were caught by actually running the pipeline end-to-end against real data (not just importing modules) — exactly the kind of thing a "re-check" is for.

---

9. MASTER PROMPT REQUIREMENTS COVERAGE

Section	Requirement	Status
5.3	Evidence Registry with all 11 required fields	✅ `evidence_registry.py`
5.4	Chunking, chunk metadata, source preservation, document relationships	✅ `chunking.py`
5.5	Hybrid retrieval: lexical (BM25) + semantic + metadata filtering; LSA described correctly as offline signal, not transformer embeddings	✅ `lexical_retrieval.py`, `semantic_retrieval.py`, `hybrid_retrieval.py`
5.6	Reranking by relevance, source quality, metadata, recency, query alignment	✅ `reranking.py`
5.7	Citation ID validation, claim validation, citation coverage, weak grounding detection	✅ `grounding.py`

5.8	Prompt injection protection; retrieved docs never become system instructions	✅ `prompt_injection.py`
5.9	Retrieval observability: query, filters, latency, result count, zero-result rate, top source, confidence	✅ `observability.py`
5.10	RAG evaluation: direct / paraphrased / ambiguous / no-answer / conflicting / unsupported	✅ `rag_evaluation.py`
5.12	If evidence unavailable, return nothing invented	✅ enforced in `hybrid_retrieval.py` (empty list, tested)

---

10. BUSINESS VALUE

- A defensible, source-quality-aware evidence base (120 documents across policy, infrastructure, market, developer, investment, and locality-deep-dive topics) is now retrievable by natural-language query, filterable by city/locality/topic/source type.
- The grounding validator is the deterministic gate Phase 9/10's LLM explanation layer must pass through — it independently catches hallucinated citations and misquoted numbers *before* they reach a user, which is the entire point of Section 5.7.
- The prompt-injection scanner protects Phase 9's future Evidence Agent from a retrieved document silently becoming an instruction to the LLM.
- Retrieval observability gives Phase 9/11 (Observability) a ready-made log format for zero-result-rate and latency monitoring in production.

---

11. KNOWN LIMITATIONS (see also `docs/limitations.md`)

1. **`ai.documents.embedding` (vector(384)) is intentionally left empty.** No transformer-embedding model was available in this offline environment to populate it. This phase's semantic signal (LSA, ≤40 components) is a different, lower-dimensional, purely linear signal — storing it in a 384-dim column reserved for a neural embedding model would be misleading, so it is kept separate. A future phase can populate `embedding` with a real sentence-transformer model without any change to this phase's retrieval interface.
2. **Chunking is a documented no-op on the current dataset** (documents average 221 characters, well under the 800-character chunk threshold). The chunker itself is implemented and unit-tested against longer synthetic text, but has not been exercised on real long-form documents (e.g. a full market report PDF) because none exist in the Phase 1 dataset yet.
3. **The corpus is small (120 documents).** BM25 and LSA both function correctly at this scale, but relevance ranking quality at 120 documents is not necessarily representative of behavior at, e.g., 50,000 documents — no claim of retrieval-quality-at-scale is made here.
4. **Grounding's claim-validation is exact-number-matching within a 1% tolerance**, not full semantic fact-checking — it catches a misquoted number but would not catch a claim that misrepresents the *meaning* of correctly-quoted numbers (e.g., citing a real "59.9" but mischaracterizing what it measures). This is a scope boundary, documented rather than silently overclaimed.
5. **Prompt-injection detection is pattern-based, not ML-based.** It will miss novel phrasings not covered by the 11 documented regex patterns. It is a first line of defense, not a complete guarantee — documented as such in `prompt_injection.py`'s docstring.
6. **RAG evaluation measures retrieval correctness, not full LLM answer quality**, because Phase 5 does not include an LLM call (that is Phase 9/10's layer). This is a legitimate, narrower evaluation scope, stated explicitly rather than implied to be more than it is.

---

12. DEPENDENCIES

- Upstream: Phase 1 (`ai.documents` schema and loaded data).
- Downstream: Phase 9's Evidence Agent (retrieval + grounding + injection-scan), Phase 9's Evidence Validator (consumes `grounding.validate_answer()` output directly), Phase 10's AI Explanation Layer (must only cite evidence that has passed grounding validation).

---

13. SECURITY CONSIDERATIONS

`db/connection.py` is unchanged from Phase 1/3 — credentials only via `DB_USER`/`DB_PASSWORD` env vars, never hardcoded. `prompt_injection.py` is itself a security control (Section 14: prompt-injection protection). No PostgreSQL instance was available in this sandbox (no network access, `psycopg2` not installed) to run the DB-path scripts directly here — the DB path is written to the same specification as Phase 3's working, previously-verified `db/connection.py` and is intended to be run and re-verified in the user's local environment where the live database exists (as Phase 3 was).

---

14. PERFORMANCE CONSIDERATIONS

Full pipeline (registry build → chunk → BM25 index → LSA fit → 8 demo retrievals → rerank → grounding checks → injection scan → 6-category RAG eval) completes in well under 2 seconds on a single CPU core for the current 120-document corpus. BM25 and LSA index-build time will need to be re-measured once the corpus grows substantially (thousands of documents) — not a concern at current scale.

---

15. PRODUCTION READINESS

Per Section 24: **Production-Oriented Prototype.** Architecture, grounding gates, and injection protection are in place and tested; a live-database run against the user's local PostgreSQL (matching Phase 3's proven working setup) is the recommended next verification step before Phase 9 integration. Retrieval-quality-at-scale, a real transformer-embedding model, and production-grade prompt-injection ML detection are explicitly out of scope for this phase.

---

16. NEXT PHASE DEPENDENCIES

Phases 6/7/8 (Financial, Risk, Geo Intelligence) may now proceed in parallel per the Master Prompt's architecture diagram — none of them strictly require Phase 5 evidence, though Phase 6/7 may optionally cite supporting evidence (e.g. a Market Outlook document) via this phase's grounding interface. Phase 9's Evidence Agent and Evidence Validator should import `src/evidence/hybrid_retrieval.py`, `reranking.py`, `grounding.py`, and `prompt_injection.py` directly rather than re-implementing retrieval logic.

---

17. FINAL STATUS

PHASE 5: COMPLETE. Evidence Registry, chunking, hybrid retrieval, reranking, grounding, prompt-injection protection, observability, and RAG evaluation all built and independently re-checked against the real 120-document dataset — 27/27 automated tests passing, 3 real bugs found and fixed during re-check. Ready for Phase 6/7/8 (parallel) and Phase 9 integration.